Departamento de Computación
FCEFQyN, Universidad Nacional de Río Cuarto
Asignatura: Análisis y Diseño de Algoritmos I
Segundo Cuatrimestre de 2025

**Trabajo Práctico No. 1**

La resolución de este trabajo práctico debe ser completada a través de GitHub Classroom, antes de las **23:55 del día 7 de Noviembre de 2025**.
El código provisto como parte de la solución a los ejercicios deberá estar documentado apropiadamente **usando Javadoc**.
Tanto la calidad de la solución, como el código y su documentación serán considerados en la calificación. Recuerde además que los trabajos prácticos **tienen defensa**, y que el trabajo en la resolución de los ejercicios debe realizarse de manera equitativa entre los miembros de cada grupo. Los grupos deben tener 3 integrantes.

1. **Pattern Frequencies**. En el contexto de aplicaciones de procesamiento de cadenas, la búsqueda de patrones es un problema que aparece con frecuencia. Mientras los algoritmos clásicos para búsqueda de patrones reciben el patrón a buscar y el texto a explorar, en diversos contextos, por ejemplo en bioinformática, resulta útil encontrar patrones directamente a partir de texto. Por ejemplo, una instancia de este problema consiste en lo siguiente:

   Dado un texto $T$, y un número natural $k$, encontrar los patrones de longitud $k$ que más se repiten en $T$.

   Notemos que hablamos de un conjunto de patrones, y no un único patrón, dado que es posible que más de un patrón tenga exactamente la misma frecuencia (es decir, el número de veces que se repite) en $T$.

   En este problema, se pide escribir un programa Java que, dado un texto $T$ y un valor $k$, encuentre el conjunto de patrones de longitud $k$, que más se repiten en $T$. Su programa debe soportar textos extensos (de varios megabytes), pero en los cuales los únicos caracteres admisibles son $A$, $C$, $G$ y $T$ (acrónimos de los cuatro tipos de bases que se pueden encontrar en cadenas de ADN). El método a implementar es $mostFrequentPatterns()$ de la clase $TextPatternUtils$. Debe respetar la interfaz provista en el proyecto, en particular retornando las excepciones que corresponden cuando los parámetros recibidos por el programa son inválidos (véase el archivo que contiene los tests para el programa). El proyecto que acompaña a este enunciado cuenta también con algunos tests para probar su implementación.

2. *Scrum* es una de las metodologías ágiles de desarrollo de software mas utilizadas por estos días. Algunos de los principios fundamentales que rigen la misma son la construcción rápida de prototipos, la entrega constante de nuevas funcionalidades y la flexibilidad para redefinir requisitos sobre la marcha. Para esto Scrum sugiere planificar ciclos cortos de desarrollo en los cuales se trata de maximizar la cantidad de funcionalidades a implementar , en función de las prioridades que el cliente le asigna a las mencionadas. Los ciclos de desarrollo (sprint en Scrum) tienen una longitud de entre 2 y 4 semanas, es decir que al cabo de ese tiempo el cliente recibirá una release del sistema con las funcionalidades que se establecieron como prioritarias y se definieron para ese sprint de desarrollo. Uno de los problemas interesantes aparejados a esta metodología es el tratar de maximizar la valoración de una release, es decir que la misma contenga el conjunto de funcionalidades (sprint backlog) que maximicen la utilidad de la release para el cliente. Cada funcionalidad a implementar requiere de recursos (tiempo de desarrollo), por lo tanto es necesario realizar un análisis de conveniencia de

cuales funcionalidades formaran parte de una release, escogiendo algunas y descartando otras, en función del valor que aporta a la release y los recursos que insume para sus desarrollo.

Scrum requiere mantener un *backlog* de producto, que consiste de una lista de características a incorporar al producto de software, acompañadas de una descripción, un valor (vinculado a la prioridad de la misma, y a la utilidad que le proveería al cliente su desarrollo), y una estimación del costo de desarrollo, e.g., en horas hombre. Un problema relevante en este contexto es la producción del *backlog* de sprint, es decir, la selección de qué características se desarrollarán en el próximo sprint, maximizando el valor global de las mismas, y respetando la duración del sprint (es decir, que de acuerdo a la estimación de costos en tiempo, pueda completarse dentro de la duración del sprint).

Se desea desarrollar, utilizando la técnica *Programación Dinámica*, una aplicación que, dado un backlog de producto, con cada funcionalidad acompañada de su valor y su estimación de costo en tiempo, y una duración para el sprint, produzca eficientemente un backlog de sprint, que maximice el valor global de las características a desarrollar, y que pueda completarse, de acuerdo a las estimaciones, en el tiempo previsto para el sprint.

Los métodos a implementar en la clase Backlog son:

- Constructor de la clase Backlog:

  ```
  public Backlog(Set<Functionality> functionalitySet, int sprintTime)
  ```

- Método que retorne el conjunto de funcionalidades óptimo para el Backlog:

  ```
  public Set<Functionality> optimalSet()
  ```

- Método que retorne el tiempo total del Backlog:

  ```
  public int getSpentTime()
  ```

- Método que retorne el valor total del Backlog:

  ```
  public int getValue()
  ```

Deberán completar también la implementación de los métodos de la clase Functionality y se debe respetar los perfiles de todos los métodos provistos. El proyecto que acompaña a este enunciado cuenta también con algunos tests para probar su implementación.